Universidad Nacional de Colombia

Facultad de Ingeniería

Estructuras de Datos

Implementación y Análisis de Complejidad de Listas, Pilas y Colas en Java

Informe de proyecto

Integrantes:
Egovail Cardozo Juan Daniel
Romero Villalba Jean Pierre
Toro Moreno Kevin Andrés

Profesor: David Alberto Herrera Álvarez
Periodo: 2026-1

Bogotá, Colombia

27 de marzo de 2026

Índice

1. Objetivo **3**

2. Descripción general del problema **3**

3. Explicación de la implementación **3**

3.1. Implementación de `List` . . . 3

3.2. Implementación de `MyStack<T>` . . . 6

3.3. Implementación de `MyQueue<T>` . . . 8

4. Métodos implementados **10**

4.1. Métodos de `List` . . . 10

4.2. Métodos de `MyStack<T>` . . . 12

4.3. Métodos de `MyQueue<T>` . . . 13

5. Análisis de complejidad **14**

5.1. Análisis teórico . . . 14

5.1.1. Complejidad teórica de `ListADT<T>` . . . 14

5.1.2. Complejidad teórica de `MyStack<T>` . . . 16

5.1.3. Complejidad teórica de `MyQueue<T>` . . . 17

5.2. Metodología experimental . . . 18

5.2.1. Estrategia general de medición . . . 19

5.2.2. Repeticiones y estadísticos reportados . . . 19

5.2.3. Configuración de las corridas . . . 20

5.2.4. Conjunto de tamaños de entrada . . . 20

5.2.5. Criterio de construcción de las pruebas . . . 20

5.2.6. Medición de operaciones elementales y compuestas . . . 21

5.2.7. Control de efectos de optimización . . . 22

5.2.8. Ejecución método a método . . . 22

5.2.9. Registro de resultados . . . 22

5.2.10. Generación de gráficas . . . 22

5.2.11. Limitaciones metodológicas . . . 23

5.3. Análisis empírico . . . 23

5.3.1. Análisis empírico de `MyStack<T>` . . . 24

5.3.2. Análisis empírico de `MyQueue<T>` . . . 26

5.3.3. Análisis de `List` singly sin cola . . . 27

5.3.4. Análisis de `List` singly con cola . . . 30

5.3.5. Análisis de `List` doubly sin cola . . . 32

5.3.6. Análisis de `List` doubly con cola . . . 35

5.3.7. Comparación transversal de operaciones de extracción 38

6. Conclusiones **39**

7. Repositorio **41**

1. Objetivo

Desarrollar e implementar distintas variantes de la estructura de datos `ListADT<T>` mediante listas enlazadas, así como las estructuras `MyStack<T>` y `MyQueue<T>` utilizando arreglos dinámicos.

Adicionalmente, analizar la complejidad de sus métodos principales tanto desde un enfoque teórico como experimental, evaluando el comportamiento de cada implementación mediante mediciones de tiempo y comparaciones gráficas.

Finalmente, establecer relaciones entre métodos equivalentes de las diferentes estructuras, identificando las implementaciones más eficientes según su desempeño observado y su complejidad esperada.

2. Descripción general del problema

Se desarrollaron:

- Cuatro variantes de `ListADT<T>` usando listas enlazadas:
  - Simplemente enlazada sin cola
  - Simplemente enlazada con cola
  - Doblemente enlazada sin cola
  - Doblemente enlazada con cola
- `MyStack<T>` usando arreglo dinámico
- `MyQueue<T>` usando arreglo dinámico

Posteriormente se realizó un análisis teórico y empírico de los métodos implementados.

3. Explicación de la implementación

3.1. Implementación de `List`

La implementación de la estructura `List` se organizó a partir de un contrato general definido mediante la interfaz `ListADT<T>`, la cual establece las operaciones fundamentales que deben soportar todas las variantes de listas desarrolladas en el proyecto, incluyendo inserciones, eliminaciones, consultas y búsqueda de elementos.

Listing 1: Contrato general definido en `ListADT<T>`

```
public interface ListADT<T>{
    void pushFront(T value);
    void pushBack(T value);
    T popFront();
    T popBack();
    boolean isEmpty();
    T topFront();
    T topBack();
    int size();
    Position<T> find(T value);
    void erase(Position<T> position);
    void addBefore(Position<T> position, T value);
    void addAfter(Position<T> position, T value);
}
```

Esta interfaz permite desacoplar la definición del comportamiento de la estructura de su implementación concreta, garantizando uniformidad entre las distintas variantes de listas enlazadas.

Una decisión de diseño clave fue la introducción de la interfaz `Position<T>`, la cual representa una abstracción de posición dentro de la lista. Esta interfaz permite que el método `find` retorne una referencia lógica al nodo, en lugar de retornar únicamente el valor.

Listing 2: Abstracción de posición usada en la lista

```
public interface Position<T>{
    T getValue();
}
```

Gracias a esta abstracción, operaciones como `erase`, `addBefore` y `addAfter` pueden trabajar directamente sobre una posición previamente obtenida, evitando ambigüedades cuando existen elementos repetidos y separando la lógica de búsqueda de la manipulación de la estructura.

Cada implementación concreta define una clase interna `Node<T>`, la cual encapsula la representación real del nodo y, al mismo tiempo, implementa la interfaz `Position<T>`.

Listing 3: Nodo interno que implementa `Position<T>`

```java
private static class Node<T> implements Position<T>{
    private T value;
    private Node<T> next;

    public Node(T value){
        this.value = value;
    }

    @Override
    public T getValue() {
        return value;
    }
}
```

Este diseño permite retornar nodos como posiciones sin exponer directamente la estructura interna de la lista, manteniendo así un mayor nivel de encapsulamiento.

A partir de esta base común, se desarrollaron cuatro variantes de listas enlazadas:

- Lista simplemente enlazada sin cola
- Lista simplemente enlazada con cola
- Lista doblemente enlazada sin cola
- Lista doblemente enlazada con cola

Las diferencias entre estas implementaciones radican principalmente en dos decisiones estructurales: la presencia o ausencia de enlaces dobles (`prev`) y el uso de una referencia al último nodo (`tail`). Estas decisiones afectan directamente la forma en que se actualizan las referencias entre nodos y, por ende, la eficiencia de las operaciones.

Por ejemplo, en listas simplemente enlazadas, la ausencia de una referencia al nodo anterior implica que operaciones como `erase` o `addBefore` requieren recorrer la lista para encontrar el predecesor del nodo objetivo. En contraste, en listas doblemente enlazadas, este acceso está disponible directamente mediante la referencia `prev`, lo que permite realizar dichas operaciones de manera más eficiente.

De manera similar, la incorporación de una referencia `tail` elimina la necesidad de recorrer la estructura para acceder al último nodo, optimizando operaciones como `pushBack` y `topBack`. Sin embargo, en listas simplemente enlazadas con cola, la eliminación del último elemento sigue requiriendo un recorrido, ya que no se dispone de acceso directo al nodo anterior.

Adicionalmente, todas las implementaciones mantienen un atributo `size`, el cual se actualiza en cada inserción y eliminación, permitiendo que la operación `size()` se ejecute en tiempo constante.

En conjunto, estas decisiones de diseño permiten construir una familia de estructuras con el mismo comportamiento externo, pero con diferencias significativas en su desempeño interno. Estas diferencias serán analizadas en detalle en la siguiente sección, donde se estudia la complejidad temporal de cada operación en función de la implementación utilizada.

3.2. Implementación de `MyStack<T>`

La estructura `MyStack<T>` fue definida mediante una interfaz que establece el comportamiento fundamental de una pila, siguiendo el principio LIFO (*Last In, First Out*). Esta interfaz permite separar el contrato de la estructura de su implementación concreta.

Listing 4: Contrato definido en `MyStack<T>`

```
public interface MyStack<T> {
    void push(T x);
    T pop();
    T peek();
    boolean isEmpty();
    int size();
    void delete(T n);
}
```

La implementación desarrollada se basó en un arreglo dinámico, encapsulado en la clase `DynamicArray<T>`. Esta decisión permite realizar inserciones, eliminaciones y consultas sobre el tope de la pila de forma eficiente, ya que todas estas operaciones se concentran en el extremo final del arreglo.

Listing 5: Uso de arreglo dinámico en `ArrayStack<T>`

```
private DynamicArray<T> array;

public ArrayStack() {
    array = new DynamicArray<>();
}
```

Las operaciones principales de la pila se implementan directamente sobre la última posición ocupada del arreglo: `push` inserta al final, `pop` elimina el último elemento y `peek` consulta dicho valor sin retirarlo.

Listing 6: Operaciones principales en `MyStack`

```java
public void push(T x) {
    array.add(x);
}

public T pop() {
    return array.removeLast();
}

public T peek() {
    return array.get(array.size() - 1);
}
```

El arreglo subyacente crece dinámicamente cuando se alcanza la capacidad máxima, mediante la creación de un nuevo arreglo de mayor tamaño y la copia de los elementos existentes. Esta estrategia permite mantener una buena eficiencia promedio en las inserciones.

Listing 7: Redimensionamiento del arreglo dinámico

```java
capacity *= 2;
T[] newData = (T[]) new Object[capacity];

for (int i = 0; i < size; i++) {
    newData[i] = data[i];
}
```

Además de las operaciones clásicas de una pila, la interfaz incluye el método `delete`, que elimina la primera aparición de un valor específico. Esta operación no forma parte del comportamiento tradicional de una pila, por lo que su implementación requiere desapilar elementos hasta encontrar el objetivo y luego reconstruir la estructura.

Listing 8: Idea general del método `delete` en `MyStack`

```java
while (!isEmpty()) {
    T value = pop();

    if (value.equals(n)) {
        break;
    }

    temp.add(value);
}
```

En conjunto, esta implementación prioriza la eficiencia de las operaciones fundamentales del modelo LIFO, mientras que operaciones adicionales como `delete` permiten extender el análisis comparativo a escenarios menos restrictivos, a costa de un mayor costo computacional.

3.3. Implementación de `MyQueue<T>`

La estructura `MyQueue<T>` fue definida mediante una interfaz que establece el comportamiento fundamental de una cola, siguiendo el principio FIFO (*First In, First Out*). Al igual que en el caso de la pila, esta interfaz permite desacoplar la definición del comportamiento de la estructura de su implementación concreta.

Listing 9: Contrato definido en `MyQueue<T>`

```
public interface MyQueue<T> {
    void enqueue(T x);
    T dequeue();
    T front();
    boolean isEmpty();
    int size();
    void delete(T n);
}
```

La implementación desarrollada se basó en un arreglo circular dinámico. Esta decisión evita el desplazamiento de elementos al insertar o eliminar, ya que la estructura mantiene dos índices, `front` y `rear`, que representan el inicio y el final lógico de la cola.

Listing 10: Estructura base de la cola circular

```
private T[] data;
private int front;
private int rear;
private int size;
private int capacity;
```

Las operaciones principales se realizan actualizando estos índices mediante aritmética modular. De este modo, `enqueue` inserta un elemento al final lógico de la cola y `dequeue` elimina el elemento ubicado al frente, reutilizando posiciones liberadas dentro del arreglo.

Listing 11: Uso de índices circulares en `MyQueue`

```
data[rear] = x;
rear = (rear + 1) % capacity;

T value = data[front];
front = (front + 1) % capacity;
```

La operación `front` consulta directamente el elemento ubicado al inicio lógico de la estructura, mientras que `isEmpty` y `size` dependen únicamente del estado interno del arreglo y de los contadores mantenidos.

Cuando la capacidad disponible se agota, se realiza un redimensionamiento dinámico. En este proceso, los elementos deben copiarse en un nuevo arreglo conservando su orden lógico a partir de la posición marcada por `front`.

Listing 12: Redimensionamiento preservando el orden lógico

```
for (int i = 0; i < size; i++) {
    newData[i] = data[(front + i) % capacity];
}

data = newData;
front = 0;
rear = size;
```

Al igual que en la pila, se implementó también el método `delete`, el cual permite eliminar la primera aparición de un valor específico. Dado que la cola solo permite acceso directo al frente, esta operación requiere recorrer la estructura mediante sucesivas llamadas a `dequeue` y reinsertar los elementos que no coinciden con el valor buscado.

Listing 13: Idea general del método `delete` en `MyQueue`

```
for (int i = 0; i < currentSize; i++) {
    T value = dequeue();

    if (!deleted && value.equals(n)) {
        deleted = true;
        continue;
    }

    enqueue(value);
}
```

En conjunto, esta implementación prioriza la eficiencia de las operaciones fundamentales del modelo FIFO, aprovechando el comportamiento circular del arreglo para evitar costos innecesarios por desplazamiento. De nuevo, el método `delete` amplía el análisis comparativo del proyecto, aunque introduce un costo mayor que las operaciones principales de la estructura.

4. Métodos implementados

4.1. Métodos de `List`

Los métodos implementados para la estructura `List` fueron definidos a través de la interfaz `ListADT<T>`, la cual establece un contrato común para todas las variantes de listas enlazadas desarrolladas en el proyecto. Esta decisión permite unificar el comportamiento esperado de la estructura, independientemente de si su implementación concreta corresponde a una lista simplemente enlazada, doblemente enlazada, con cola o sin cola.

Listing 14: Métodos declarados en `ListADT<T>`

```java
public interface ListADT<T>{
    void pushFront(T value);
    void pushBack(T value);
    T popFront();
    T popBack();
    boolean isEmpty();
    T topFront();
    T topBack();
    int size();
    Position<T> find(T value);
    void erase(Position<T> position);
    void addBefore(Position<T> position, T value);
    void addAfter(Position<T> position, T value);
}
```

A continuación, se describe la finalidad de cada uno de los métodos definidos en este contrato, así como el motivo de su inclusión dentro de la interfaz.

- `pushFront(T value)`: inserta un nuevo elemento al inicio de la lista. Su firma no retorna ningún valor, ya que su propósito es modificar directamente la estructura agregando un nuevo nodo en la primera posición.
- `pushBack(T value)`: inserta un nuevo elemento al final de la lista. Al igual que `pushFront`, no retorna ningún valor, puesto que su función es actualizar la estructura enlazando el nuevo nodo al extremo final.

- `popFront()`: elimina el primer elemento de la lista y retorna su valor. Se definió con retorno de tipo `T` para que, además de modificar la estructura, permita recuperar el elemento removido.
- `popBack()`: elimina el último elemento de la lista y retorna su valor. Su inclusión permite trabajar de forma simétrica con ambos extremos de la estructura, especialmente útil al comparar implementaciones con y sin referencia al final.
- `isEmpty()`: retorna un valor booleano que indica si la lista se encuentra vacía. Este método es importante como operación de apoyo para validar el estado de la estructura antes de ejecutar inserciones, eliminaciones o consultas.
- `topFront()`: retorna el valor almacenado en el primer nodo sin eliminarlo. Se utiliza como operación de consulta sobre el extremo inicial de la lista.
- `topBack()`: retorna el valor almacenado en el último nodo sin eliminarlo. Al igual que `topFront`, es una operación de consulta, pero orientada al extremo final.
- `size()`: retorna la cantidad de elementos almacenados en la lista. Se definió con retorno entero para consultar el tamaño actual de la estructura sin necesidad de recorrerla.
- `find(T value)`: busca un elemento dentro de la lista y retorna una referencia de tipo `Position<T>` asociada al nodo donde se encuentra dicho valor. Esta decisión de diseño permite separar la búsqueda del acceso directo al nodo, facilitando operaciones posteriores sobre posiciones concretas.
- `erase(Position<T> position)`: elimina de la lista el nodo asociado a una posición previamente obtenida. El uso de `Position<T>` en lugar de un valor directo permite eliminar específicamente un nodo identificado por la estructura, evitando ambigüedades cuando existen elementos repetidos.
- `addBefore(Position<T> position, T value)`: inserta un nuevo elemento antes de una posición dada. Este método permite realizar inserciones relativas a un nodo ya existente, lo cual extiende la flexibilidad de la estructura más allá de los extremos de la lista.
- `addAfter(Position<T> position, T value)`: inserta un nuevo elemento después de una posición dada. Al igual que `addBefore`, su diseño responde a la necesidad de manipular la estructura en posiciones internas y no únicamente en el inicio o en el final.

En conjunto, estos métodos permiten cubrir las operaciones fundamentales de inserción, eliminación, consulta, búsqueda y modificación relativa de posiciones dentro de la lista. La interfaz `ListADT<T>` fue diseñada de esta manera para que todas las implementaciones

compartieran exactamente el mismo comportamiento externo, dejando que las diferencias entre ellas surgieran únicamente en la forma de resolver internamente cada operación y, por tanto, en su complejidad temporal.

4.2. Métodos de `MyStack<T>`

Los métodos implementados para la estructura `MyStack<T>` fueron definidos mediante una interfaz que establece el comportamiento fundamental de una pila. Esta abstracción permite describir las operaciones esenciales asociadas al modelo LIFO (*Last In, First Out*), en el cual el último elemento insertado es el primero en ser removido.

Listing 15: Métodos declarados en `MyStack<T>`

```
public interface MyStack<T> {
    void push(T x);
    T pop();
    T peek();
    boolean isEmpty();
    int size();
    void delete(T n);
}
```

A continuación, se describe la finalidad de cada uno de los métodos definidos en esta interfaz.

- `push(T x)`: inserta un nuevo elemento en el tope de la pila. No retorna ningún valor, ya que su objetivo es modificar directamente la estructura agregando un nuevo elemento en la posición más alta.
- `pop()`: elimina el elemento ubicado en el tope de la pila y retorna su valor. Este método materializa el principio LIFO, puesto que siempre remueve el último elemento insertado.
- `peek()`: retorna el valor del elemento ubicado en el tope de la pila sin eliminarlo. Su inclusión permite consultar el estado inmediato de la estructura sin modificar su contenido.
- `isEmpty()`: retorna un valor booleano que indica si la pila se encuentra vacía. Este método es útil para validar el estado de la estructura antes de ejecutar operaciones como `pop()` o `peek()`.
- `size()`: retorna la cantidad de elementos almacenados en la pila. Permite conocer el tamaño actual de la estructura sin necesidad de recorrerla.

- `delete(T n)`: elimina la primera aparición del valor indicado dentro de la pila. Aunque esta no es una operación clásica en una pila abstracta, fue incluida en el contrato del proyecto para permitir comparaciones adicionales entre estructuras y analizar el costo de eliminar un elemento específico dentro de una organización LIFO.

En conjunto, estos métodos cubren tanto las operaciones fundamentales de una pila como una operación adicional de eliminación por valor. La interfaz `MyStack<T>` fue diseñada de esta manera para proporcionar un contrato uniforme sobre el cual pudiera construirse una implementación concreta y, posteriormente, analizar su comportamiento teórico y empírico.

4.3. Métodos de `MyQueue<T>`

Los métodos implementados para la estructura `MyQueue<T>` fueron definidos mediante una interfaz que establece el comportamiento fundamental de una cola. Esta abstracción responde al modelo FIFO (*First In, First Out*), en el cual el primer elemento en ingresar a la estructura es también el primero en ser removido.

Listing 16: Métodos declarados en `MyQueue<T>`

```
public interface MyQueue<T> {
    void enqueue(T x);
    T dequeue();
    T front();
    boolean isEmpty();
    int size();
    void delete(T n);
}
```

A continuación, se describe la finalidad de cada uno de los métodos definidos en esta interfaz.

- `enqueue(T x)`: inserta un nuevo elemento al final lógico de la cola. No retorna ningún valor, ya que su función es modificar la estructura agregando un nuevo elemento en la posición de entrada.
- `dequeue()`: elimina el elemento ubicado al frente de la cola y retorna su valor. Este método refleja el comportamiento FIFO, puesto que siempre remueve el elemento que lleva más tiempo dentro de la estructura.
- `front()`: retorna el valor del elemento ubicado al frente de la cola sin eliminarlo. Se trata de una operación de consulta que permite inspeccionar el siguiente elemento a ser removido.

- `isEmpty()`: retorna un valor booleano que indica si la cola se encuentra vacía. Este método resulta útil para validar el estado de la estructura antes de ejecutar operaciones como `dequeue()` o `front()`.
- `size()`: retorna la cantidad de elementos almacenados en la cola. Permite conocer el tamaño actual de la estructura sin necesidad de recorrer sus elementos.
- `delete(T n)`: elimina la primera aparición del valor indicado dentro de la cola. Al igual que en la pila, esta no corresponde a una operación elemental del modelo abstracto clásico, pero fue incorporada en el contrato del proyecto con el propósito de extender el análisis de comportamiento y comparar el costo de eliminar un valor específico en una estructura FIFO.

En conjunto, estos métodos cubren las operaciones esenciales de una cola y una operación adicional de eliminación por valor. La interfaz `MyQueue<T>` define así un contrato uniforme que permite implementar la estructura de manera concreta y, posteriormente, estudiar su comportamiento desde un punto de vista teórico y experimental.

5. Análisis de complejidad

5.1. Análisis teórico

5.1.1. Complejidad teórica de `ListADT<T>`

El análisis de complejidad teórica de la estructura `ListADT<T>` se realizó considerando las cuatro variantes implementadas: lista simplemente enlazada sin cola (`Singly`), lista simplemente enlazada con cola (`SinglyTail`), lista doblemente enlazada sin cola (`Doubly`) y lista doblemente enlazada con cola (`DoublyTail`).

Dado que todas las implementaciones comparten el mismo contrato, las diferencias en complejidad provienen exclusivamente de su representación interna, en particular del uso de la referencia `tail` y de la presencia de enlaces dobles mediante `prev`.

En la Tabla 1 se presenta la complejidad temporal esperada para cada uno de los métodos definidos en la interfaz, suponiendo que se dispone directamente de la posición requerida en los métodos que operan sobre `Position<T>`.

Cuadro 1: Complejidad teórica de los métodos de `ListADT<T>` según implementación

Método	**Singly**	**SinglyTail**	**Doubly**	**DoublyTail**
`pushFront`	$O(1)$	$O(1)$	$O(1)$	$O(1)$
`pushBack`	$O(n)$	$O(1)$	$O(n)$	$O(1)$
`popFront`	$O(1)$	$O(1)$	$O(1)$	$O(1)$
`popBack`	$O(n)$	$O(n)$	$O(n)$	$O(1)$
`topFront`	$O(1)$	$O(1)$	$O(1)$	$O(1)$
`topBack`	$O(n)$	$O(1)$	$O(n)$	$O(1)$
`isEmpty`	$O(1)$	$O(1)$	$O(1)$	$O(1)$
`size`	$O(1)$	$O(1)$	$O(1)$	$O(1)$
`find`	$O(n)$	$O(n)$	$O(n)$	$O(n)$
`erase`*	$O(n)$	$O(n)$	$O(1)$	$O(1)$
`addBefore`*	$O(n)$	$O(n)$	$O(1)$	$O(1)$
`addAfter`*	$O(1)$	$O(1)$	$O(1)$	$O(1)$

**Las complejidades indicadas corresponden al costo de la operación suponiendo que se dispone previamente de una referencia válida de tipo `Position<T>`.*

A partir de la tabla anterior, se observa que las operaciones sobre el inicio de la lista, como `pushFront`, `popFront` y `topFront`, presentan complejidad constante en todas las implementaciones, debido a que siempre existe acceso directo al primer nodo mediante la referencia `head`.

En contraste, las operaciones sobre el extremo final dependen de la existencia de una referencia `tail`. En las implementaciones sin cola (`Singly` y `Doubly`), operaciones como `pushBack` y `topBack` requieren recorrer la lista desde `head` hasta el último nodo, lo que implica un costo lineal. En las implementaciones con cola, este recorrido se evita, permitiendo resolver dichas operaciones en tiempo constante.

El caso de `popBack` evidencia una diferencia adicional entre las estructuras. En `SinglyTail`, a pesar de contar con una referencia `tail`, no es posible eliminar el último nodo en tiempo constante, ya que no se dispone de acceso directo al nodo anterior. Por ello, es necesario recorrer la lista hasta el penúltimo nodo. En cambio, en `DoublyTail`, la presencia de la referencia `prev` permite acceder directamente al nodo anterior a `tail`, logrando una complejidad constante.

Por otro lado, el método `find` presenta complejidad lineal en todas las implementaciones, dado que la búsqueda se realiza de manera secuencial desde `head` hasta encontrar el elemento o alcanzar el final de la lista.

Los métodos `erase`, `addBefore` y `addAfter` requieren un análisis particular. Cuando se dispone de una referencia directa al nodo objetivo, `addAfter` puede resolverse en tiempo constante en todas las implementaciones, ya que solo implica modificar enlaces locales. En el caso de `erase` y `addBefore`, las listas doblemente enlazadas permiten realizar estas operaciones en tiempo constante gracias a la referencia `prev`, mientras que en las listas simplemente enlazadas es necesario ubicar el nodo anterior mediante un recorrido adicional.

No obstante, en el uso práctico de la estructura, es frecuente que la posición sobre la cual se desea operar no esté disponible previamente, por lo que debe obtenerse mediante una llamada a `find`. Esto introduce un *overhead* adicional que afecta el costo total de la operación.

Con el fin de reflejar este escenario más cercano al uso real, en la Tabla 2 se presenta la complejidad esperada al considerar el flujo completo que incluye la búsqueda previa.

Cuadro 2: Complejidad total considerando búsqueda previa con `find`

Operación compuesta	**Singly**	**SinglyTail**	**Doubly**	**DoublyTail**
`find + erase`	$O(n)$	$O(n)$	$O(n)$	$O(n)$
`find + addBefore`	$O(n)$	$O(n)$	$O(n)$	$O(n)$
`find + addAfter`	$O(n)$	$O(n)$	$O(n)$	$O(n)$

De acuerdo con lo anterior, aunque ciertas operaciones pueden ejecutarse en tiempo constante cuando se dispone de la posición objetivo, el costo total en escenarios reales suele estar dominado por el tiempo de búsqueda. Este fenómeno resulta especialmente relevante en el análisis empírico, donde la medición del tiempo de ejecución debe considerar este *overhead* para reflejar de manera fiel el comportamiento de la estructura en situaciones prácticas.

5.1.2. Complejidad teórica de `MyStack<T>`

La estructura `MyStack<T>` fue implementada mediante un arreglo dinámico, lo que permite realizar operaciones sobre el tope de la pila de forma eficiente. Dado que la pila sigue el principio LIFO (*Last In, First Out*), todas las operaciones principales se concentran en el extremo final del arreglo.

En la Tabla 3 se presenta la complejidad temporal esperada de los métodos definidos en la interfaz.

Cuadro 3: Complejidad teórica de los métodos de `MyStack<T>`

Método	**Complejidad**
`push`	$O(1)$ amortizado
`pop`	$O(1)$
`peek`	$O(1)$
`isEmpty`	$O(1)$
`size`	$O(1)$
`delete`	$O(n)$

Las operaciones `push`, `pop` y `peek` se realizan directamente sobre el último elemento del arreglo dinámico, por lo que su costo es constante. Sin embargo, la operación `push` se clasifica como $O(1)$ amortizado, ya que en situaciones donde el arreglo alcanza su capacidad máxima, es necesario redimensionarlo, lo que implica copiar todos los elementos a un nuevo arreglo de mayor tamaño.

Las operaciones `isEmpty` y `size` también se resuelven en tiempo constante, ya que dependen únicamente de variables internas que mantienen el estado de la estructura.

Por otro lado, el método `delete` introduce un comportamiento distinto al de una pila clásica. Para eliminar un elemento específico, es necesario desapilar elementos hasta encontrar el valor buscado y luego reconstruir la pila con los elementos restantes. Este proceso implica recorrer una cantidad proporcional de elementos, por lo que su complejidad en el peor caso es $O(n)$.

En consecuencia, la implementación de `MyStack<T>` ofrece un desempeño eficiente en sus operaciones fundamentales, mientras que operaciones adicionales como `delete` presentan un costo mayor debido a las restricciones inherentes al modelo LIFO.

5.1.3. Complejidad teórica de `MyQueue<T>`

La estructura `MyQueue<T>` fue implementada mediante un arreglo circular dinámico, lo que permite optimizar el uso del espacio y evitar desplazamientos de elementos durante las operaciones. Esta estructura sigue el principio FIFO (*First In, First Out*), en el cual el primer elemento en ingresar es el primero en salir.

En la Tabla 4 se presenta la complejidad temporal esperada de los métodos definidos en la interfaz.

Cuadro 4: Complejidad teórica de los métodos de `MyQueue<T>`

Método	**Complejidad**
`enqueue`	$O(1)$ amortizado
`dequeue`	$O(1)$
`front`	$O(1)$
`isEmpty`	$O(1)$
`size`	$O(1)$
`delete`	$O(n)$

Las operaciones principales de la cola, `enqueue` y `dequeue`, se realizan mediante el uso de índices circulares (`rear` y `front`), lo que permite insertar y eliminar elementos sin necesidad de desplazar el contenido del arreglo. Esto garantiza un costo constante para estas operaciones en condiciones normales.

Al igual que en el caso de la pila, la operación `enqueue` es de tipo amortizado debido al proceso de redimensionamiento del arreglo cuando se alcanza la capacidad máxima. Durante este proceso, los elementos deben copiarse a un nuevo arreglo manteniendo su orden lógico, lo que introduce un costo puntual de $O(n)$.

Las operaciones `front`, `isEmpty` y `size` se ejecutan en tiempo constante, ya que dependen únicamente de accesos directos a posiciones del arreglo o a variables internas.

El método `delete`, al igual que en la pila, no corresponde a una operación clásica del modelo FIFO. Su implementación requiere recorrer la cola completa mediante sucesivas operaciones de `dequeue` y `enqueue`, lo que implica un número de operaciones proporcional al tamaño de la estructura. Por esta razón, su complejidad es $O(n)$.

En términos generales, la implementación de `MyQueue<T>` mediante arreglo circular ofrece un desempeño eficiente en sus operaciones fundamentales, evitando costos adicionales asociados al desplazamiento de elementos, mientras que las operaciones no estándar como `delete` presentan un costo lineal.

5.2. Metodología experimental

Con el fin de contrastar el análisis teórico de complejidad con el comportamiento observado en la práctica, se diseñó una metodología experimental basada en la medición del tiempo de ejecución de operaciones sobre listas enlazadas, pilas y colas implementadas en Java. La estrategia final de medición se ajustó durante el desarrollo del proyecto para mejorar la consistencia de los resultados, especialmente en operaciones de orden constante, evitando

la interferencia producida al ejecutar toda la suite de benchmarks dentro de una misma corrida de la JVM. Por esta razón, la versión final de los resultados se obtuvo ejecutando los benchmarks de forma individual, método por método.

5.2.1. Estrategia general de medición

La medición experimental se implementó mediante clases específicas por familia de estructura: `ListBenchmark`, `StackBenchmark` y `QueueBenchmark`. Estas clases delegan la ejecución repetida de cada prueba a la clase `BenchmarkRunner`, encargada de realizar corridas de calentamiento, repeticiones medidas y cálculo de estadísticas descriptivas.

Cada medición individual se realizó en nanosegundos usando `System.nanoTime()`, encapsulado en la utilidad `Timer.measure(...)`.

Listing 17: Medición de tiempo con `System.nanoTime()`

```
public static long measure(Runnable task) {
    long start = System.nanoTime();
    task.run();
    long end = System.nanoTime();
    return end - start;
}
```

Se trabajó en nanosegundos porque varias de las operaciones estudiadas, en particular las operaciones de orden constante, presentan tiempos de ejecución muy pequeños.

5.2.2. Repeticiones y estadísticos reportados

Para cada tamaño de entrada y cada operación se realizaron múltiples ejecuciones independientes. En la configuración final del benchmark se usaron:

- 3 corridas de calentamiento (*warmup*),
- 5 corridas medidas por cada tamaño de entrada.

La clase `BenchmarkRunner` centraliza este proceso y calcula promedio, mediana, mínimo y máximo para cada conjunto de muestras.

Listing 18: Esquema general de repeticiones medidas

```
for (int i = 0; i < measuredRuns; i++) {
    long value = measuredRun.getAsLong();
    samples[i] = value;
    sum += value;
}
```

Aunque en los archivos CSV se almacenaron promedio, mediana, mínimo y máximo, para el análisis comparativo y la mayoría de las gráficas se privilegió la **mediana**. Esta decisión se tomó porque, en operaciones muy pequeñas, el promedio resultó más sensible a variaciones puntuales del entorno de ejecución, mientras que la mediana ofreció una representación más estable del comportamiento típico.

5.2.3. Configuración de las corridas

La configuración del número de warmups y repeticiones se controla desde `BenchmarkRunner` mediante propiedades del sistema:

Listing 19: Configuración dinámica de warmups y repeticiones

```
public static int warmupRuns() {
    return Integer.parseInt(System.getProperty("benchmark.warmup"
        , "3"));
}

public static int measuredRuns() {
    return Integer.parseInt(System.getProperty("benchmark.
        repetitions", "5"));
}
```

Esto permitió mantener una configuración única y consistente para todas las estructuras, y al mismo tiempo conservar flexibilidad para pruebas preliminares o de depuración.

5.2.4. Conjunto de tamaños de entrada

En la versión final de los experimentos se utilizó un único conjunto de tamaños de entrada para todas las operaciones benchmarkeadas:

$$10,\ 10^2,\ 10^3,\ 10^4,\ 10^5,\ 10^6,\ 10^7,\ 10^8$$

Esta selección permite observar la evolución temporal de las operaciones a lo largo de varios órdenes de magnitud, manteniendo una progresión simple y uniforme.

5.2.5. Criterio de construcción de las pruebas

Cada operación se midió sobre una estructura previamente preparada hasta tamaño n, construida **fuera** del bloque cronometrado. Es decir, no se midió el costo acumulado de construir progresivamente la estructura, sino el costo de ejecutar una operación cuando la estructura ya se encontraba en un estado representativo de tamaño n.

En el caso de las listas, la precarga se realizó antes de iniciar el cronómetro:

Listing 20: Precarga de la estructura antes de medir

```
private static void preloadForReadWriteOps(ListADT<Integer> list,
    int n) {
    for (int i = 0; i < n; i++) {
        list.pushFront(i);
    }
}
```

De forma análoga, en pilas y colas se insertaron previamente n elementos antes de medir operaciones como `push`, `pop`, `peek`, `enqueue`, `dequeue` o `front`. Esto garantiza que la medición represente el costo de operar sobre una estructura de tamaño n, y no el tiempo promedio de una secuencia de tamaños crecientes.

5.2.6. Medición de operaciones elementales y compuestas

La versión final del benchmark distingue entre operaciones elementales y operaciones compuestas:

- **Operaciones elementales:** se mide únicamente la operación sobre la estructura ya construida. Por ejemplo, `pushFront`, `popFront`, `pushBack`, `popBack`, `peek`, `front`, `find`, etc.
- **Operaciones compuestas:** cuando una operación requiere un nodo o posición previa, dicha referencia se obtiene antes de iniciar el cronómetro y el tiempo medido corresponde únicamente a la modificación posterior.

En listas, esto aplica a `erase`, `addBefore` y `addAfter`. En la metodología final, el nodo objetivo se obtiene mediante `find` **antes** de iniciar la medición, por lo que el tiempo registrado corresponde a la operación de modificación sobre la `Position` ya obtenida.

Listing 21: Ejemplo de medición de `addAfter` con `Position` ya obtenida

```
Position<Integer> addAfterTarget = list.find(targetValue);
return Timer.measure(() -> {
    if (addAfterTarget != null) {
        list.addAfter(addAfterTarget, -1);
    }
});
```

Esta decisión metodológica fue importante porque permitió separar el costo de búsqueda del costo de modificación, haciendo más coherente la comparación entre implementaciones con distinta estructura interna.

En contraste, los métodos `delete` de pila y cola sí corresponden a operaciones compuestas internas que recorren la estructura, por lo que su comportamiento empírico se interpreta como lineal.

5.2.7. Control de efectos de optimización

Para las operaciones que devuelven un valor, en la versión final del benchmark se almacenó el resultado en una variable auxiliar compartida. Esto evita que lecturas como `peek`, `front`, `popFront` o `dequeue` queden completamente sin efecto observable dentro del benchmark, mejorando la estabilidad de las mediciones de operaciones de corta duración.

5.2.8. Ejecución método a método

Durante la validación experimental se observó que ejecutar toda la suite completa dentro de una sola corrida introducía variabilidad adicional, especialmente en operaciones $O(1)$, debido a presión de memoria, recolección de basura y efectos acumulados del entorno de ejecución. Por ello, los resultados finales se generaron ejecutando los benchmarks **método a método**, de manera independiente, permitiendo que cada corrida escribiera su respectivo archivo CSV sin arrastrar el estado de pruebas anteriores.

Esta estrategia no cambia la naturaleza del benchmark, pero sí reduce la interferencia entre mediciones y mejora la consistencia empírica de las operaciones pequeñas.

5.2.9. Registro de resultados

Los resultados de cada benchmark se almacenaron en archivos CSV independientes por estructura y por operación con el formato:

```
size,avg_time_ns,median_ns,min_ns,max_ns
```

Esto permitió separar la etapa de medición de la etapa de análisis y graficación, y además facilitó repetir selectivamente benchmarks individuales sin necesidad de recalcular toda la suite.

5.2.10. Generación de gráficas

La visualización de resultados se realizó posteriormente mediante scripts en Python usando `pandas` y `matplotlib`. Esta etapa se ejecutó fuera del proceso de medición para evitar introducir *overhead* en los tiempos registrados.

En las gráficas comparativas se emplearon escalas logarítmicas en el eje de entrada y, cuando fue útil para comparar operaciones de distinto orden de crecimiento, también en el eje de tiempo. Esta elección permitió visualizar con mayor claridad tanto operaciones contenidas como operaciones de crecimiento lineal dentro de una misma figura.

5.2.11. Limitaciones metodológicas

Aunque la metodología permite identificar de forma clara tendencias de crecimiento y diferencias entre implementaciones, los resultados deben interpretarse como mediciones empíricas sujetas al entorno de ejecución de la JVM, al manejo de memoria y a la variabilidad propia de operaciones muy pequeñas. Por ello, en operaciones $O(1)$ pueden observarse incrementos moderados en tamaños extremos sin que ello implique un cambio de complejidad algorítmica.

En consecuencia, las conclusiones del trabajo se apoyan principalmente en la comparación entre tendencias, órdenes de magnitud relativos y contrastes entre operaciones constantes y lineales, más que en la expectativa de tiempos absolutamente planos para todas las operaciones pequeñas..

5.3. Análisis empírico

En esta sección se presentan los resultados obtenidos a partir de los benchmarks realizados sobre las estructuras `List`, `MyStack<T>` y `MyQueue<T>`. El propósito es analizar el comportamiento observado en la práctica y contrastarlo con el análisis teórico de complejidad desarrollado en las secciones anteriores.

Los valores presentados en tablas y gráficas corresponden a los resultados generados mediante el framework de benchmarking descrito en la metodología experimental. Para cada operación y cada tamaño de entrada se registraron múltiples ejecuciones, a partir de las cuales se calcularon promedio, mediana, mínimo y máximo. En el análisis comparativo se privilegia la **mediana**, ya que ofrece una representación más estable del comportamiento típico de cada operación, especialmente en aquellas de corta duración donde el promedio puede verse más afectado por variaciones puntuales del entorno de ejecución.

Es importante resaltar que los tiempos reportados corresponden al costo de ejecutar operaciones sobre estructuras previamente construidas hasta tamaño n. En consecuencia, las gráficas no deben interpretarse como una medición acumulada del costo de construir toda la estructura paso a paso, sino como la medición del comportamiento de una operación específica cuando actúa sobre una instancia de tamaño controlado. En algunos casos particulares, como `delete` en pila y cola, el método evaluado implica internamente un recorrido de la estructura, lo cual se refleja de manera directa en el crecimiento observado.

A partir de esta base, el análisis empírico se organiza en varias partes. En primer lugar, se estudia el comportamiento experimental de `MyStack<T>` y `MyQueue<T>` de forma individual. Posteriormente, se comparan ambas estructuras entre sí mediante operaciones equivalentes. Finalmente, se analiza el comportamiento de las distintas implementaciones de `List`, tanto desde la comparación interna de sus métodos como a través de compara-

ciones entre implementaciones.

5.3.1. Análisis empírico de `MyStack<T>`

La Figura 1 presenta la comparación conjunta de los cuatro métodos evaluados en la implementación de pila: `push`, `pop`, `peek` y `delete`. Esta representación resulta especialmente útil porque permite observar, en una sola gráfica, la diferencia entre las operaciones de costo constante y la operación de costo lineal dentro de la misma estructura.

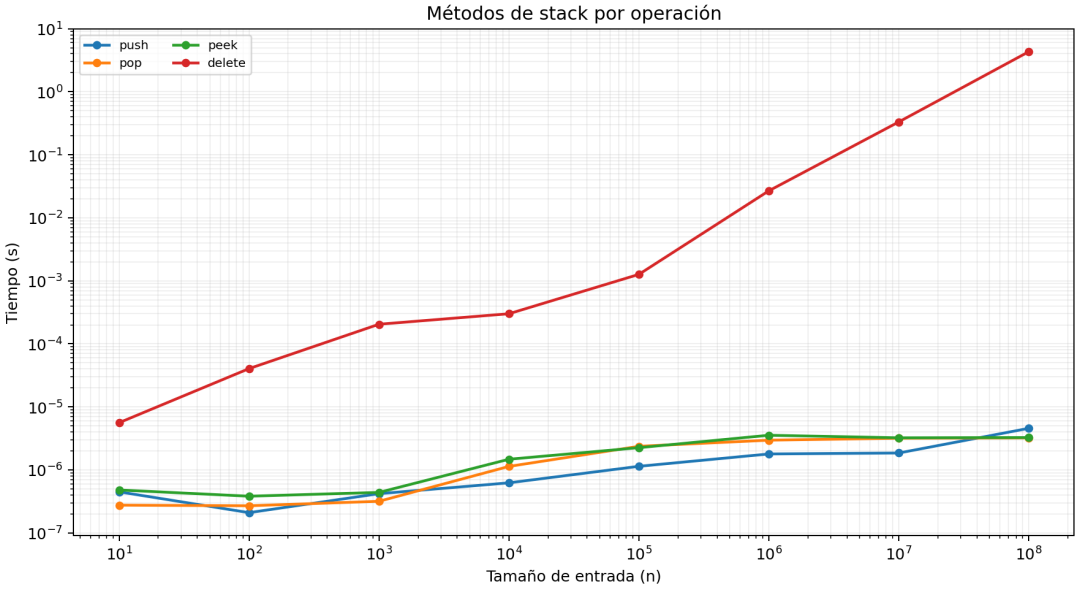

Figura 1: Comparación de métodos en la implementación `MyStack`.

Como se observa en la figura, los métodos `push`, `pop` y `peek` permanecen agrupados en la parte baja de la gráfica a lo largo de todos los tamaños de entrada. Aunque sus tiempos no son completamente planos y muestran un incremento moderado en tamaños grandes, su crecimiento se mantiene dentro de la misma escala de tiempo y no presenta el patrón de aumento pronunciado característico de una operación lineal. En términos empíricos, esto es consistente con el comportamiento esperado de operaciones de orden constante en una implementación basada en arreglo dinámico.

Por el contrario, el método `delete` se separa claramente del resto desde tamaños pequeños y su tiempo aumenta de manera muy marcada conforme crece n. Mientras `push`, `pop` y `peek` se mantienen en el orden de cientos o pocos miles de nanosegundos, `delete` pasa de miles de nanosegundos en $n = 10$ a miles de millones de nanosegundos en $n = 10^8$. Esta diferencia de varios órdenes de magnitud respalda experimentalmente que `delete` tiene un comportamiento lineal.

La razón de este resultado coincide con la implementación del método. Para eliminar un valor dado, la pila extrae elementos sucesivamente hasta encontrarlo y luego recons-

truye la estructura con los elementos restantes. En el benchmark realizado, se midió `delete(0)` sobre una pila precargada con valores desde 0 hasta $n-1$, lo que obliga a recorrer prácticamente toda la estructura antes de alcanzar el elemento buscado. Por ello, el comportamiento observado corresponde a un caso cercano al peor escenario de ejecución.

La Tabla 5 resume los tiempos medianos obtenidos para cada uno de los métodos evaluados. Se emplea la mediana como estadístico principal porque ofrece una representación más estable del comportamiento típico de cada operación, especialmente en operaciones pequeñas donde el promedio puede verse más afectado por variaciones puntuales del entorno de ejecución.

Cuadro 5: Tiempos medianos de ejecución para los métodos de `MyStack`.

Tamaño n	**push (ns)**	**pop (ns)**	**peek (ns)**	**delete (ns)**
10	449	276	481	5613
100	210	270	383	40495
1000	420	318	440	204773
10000	624	1137	1477	300513
100000	1141	2370	2254	1265657
1000000	1792	2964	3550	26779856
10000000	1855	3184	3234	331814606
100000000	4569	3220	3265	4310084192

Tomando como referencia la mediana, se observa que `peek` pasa de 481 ns en $n = 10$ a 3265 ns en $n = 10^8$, `pop` pasa de 276 ns a 3220 ns, y `push` de 449 ns a 4569 ns. En contraste, `delete` pasa de 5613 ns a $4{,}31 \times 10^9$ ns. Esta diferencia resume claramente la separación entre operaciones $O(1)$ y $O(n)$ dentro del stack.

Es importante señalar que, aunque `push`, `pop` y `peek` muestran incrementos en tamaños extremos, la comparación directa con `delete` demuestra que estas variaciones no alteran su naturaleza asintótica. Es decir, existe una degradación empírica moderada atribuible al entorno de ejecución, al manejo de memoria y a la JVM, pero no una transición hacia un comportamiento lineal. La gráfica conjunta es especialmente útil para evidenciar esto, ya que pone en contexto los pequeños incrementos de las operaciones constantes frente al crecimiento dominante de la operación lineal.

En conclusión, el análisis experimental de la implementación `MyStack` es coherente con el análisis teórico. Los métodos `push`, `pop` y `peek` exhiben un comportamiento empírico contenido y consistente con tiempo constante, mientras que `delete` presenta un crecimiento pronunciado compatible con tiempo lineal.

5.3.2. Análisis empírico de `MyQueue<T>`

La Figura 2 presenta la comparación conjunta de los cuatro métodos evaluados en la implementación de cola: `enqueue`, `dequeue`, `front` y `delete`. Al igual que en el análisis de la pila, esta representación permite contrastar en una sola gráfica las operaciones de costo constante frente a la operación de costo lineal dentro de una misma estructura.

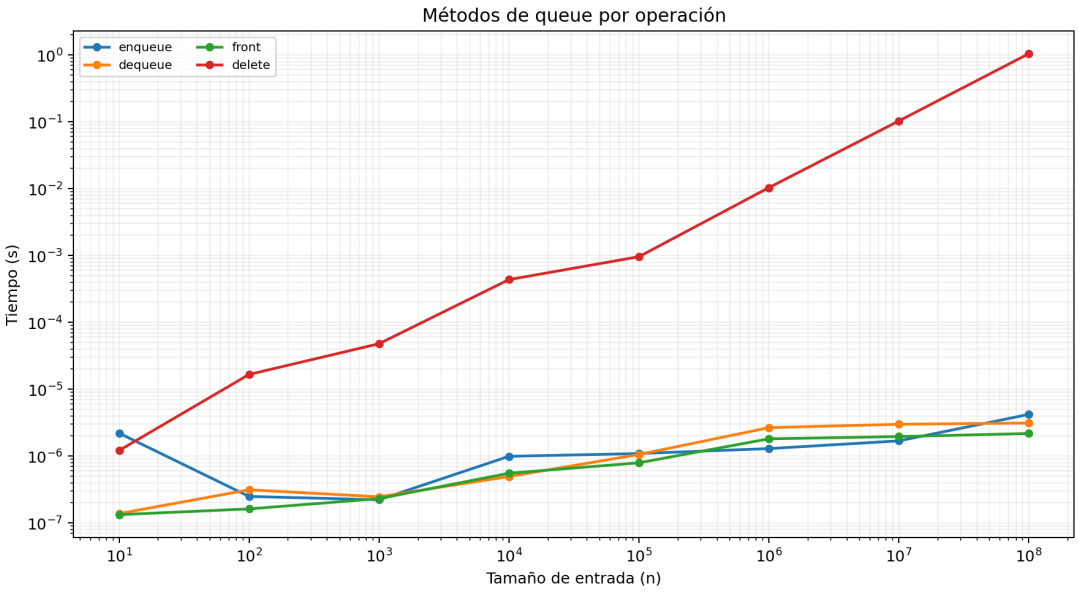

Figura 2: Comparación de métodos en la implementación `MyQueue`.

Como se observa en la figura, los métodos `enqueue`, `dequeue` y `front` permanecen agrupados en la zona inferior de la gráfica para todos los tamaños de entrada. Aunque sus tiempos no son perfectamente constantes y presentan incrementos moderados a partir de tamaños grandes, se mantienen dentro de una misma escala de tiempo y no muestran el patrón de crecimiento pronunciado propio de una operación lineal. En términos empíricos, esto es consistente con el comportamiento esperado de operaciones $O(1)$ en una cola circular implementada sobre arreglo.

En contraste, el método `delete` se separa claramente del resto y exhibe un aumento fuerte y sostenido conforme crece n. Mientras `enqueue`, `dequeue` y `front` se mantienen en el orden de cientos o pocos miles de nanosegundos, `delete` pasa de miles de nanosegundos en $n = 10$ a más de 10^9 nanosegundos en $n = 10^8$. Esta diferencia de varios órdenes de magnitud respalda experimentalmente el comportamiento lineal del método.

Este resultado coincide con la lógica de implementación. Para eliminar un valor específico, la cola debe recorrer sus elementos mediante una secuencia de extracciones e inserciones hasta localizar el elemento buscado y reconstruir el orden de la estructura. En el benchmark se midió `delete(0)` sobre una cola precargada con valores desde 0 hasta $n - 1$. Aunque el valor a eliminar se encuentra al frente, la implementación sigue recorriendo y

reorganizando la estructura durante la operación completa, lo que explica el crecimiento empírico observado.

La Tabla 6 resume los tiempos medianos de ejecución para cada método. Al igual que en las demás estructuras, se privilegia la mediana porque representa de forma más estable el comportamiento típico de cada operación y reduce la influencia de valores atípicos.

Cuadro 6: Tiempos medianos de ejecución para los métodos de `MyQueue`.

Tamaño n	**enqueue (ns)**	**dequeue (ns)**	**front (ns)**	**delete (ns)**
10	2203	138	133	1213
100	249	313	162	16643
1000	222	246	231	47931
10000	992	495	554	436748
100000	1091	1056	793	960469
1000000	1294	2663	1813	10365475
10000000	1688	2994	1959	103024348
100000000	4204	3121	2175	1044839797

Tomando como referencia la mediana, se observa que `front` pasa de 133 ns en $n = 10$ a 2175 ns en $n = 10^8$, `dequeue` pasa de 138 ns a 3121 ns, y `enqueue` pasa de 2203 ns a 4204 ns. En contraste, `delete` pasa de 1213 ns a $1{,}04 \times 10^9$ ns. Esta diferencia resume con claridad la separación entre las operaciones $O(1)$ y la operación $O(n)$ dentro de la cola.

También es importante notar que `enqueue`, `dequeue` y `front` muestran ligeros incrementos en tamaños extremos, especialmente a partir de 10^6. Sin embargo, estos aumentos permanecen contenidos dentro de una misma escala y no alteran la interpretación asintótica de las operaciones. La comparación directa con `delete` permite ver con claridad que dichos incrementos responden más al costo constante propio de la implementación y al entorno de ejecución que a un cambio real hacia un comportamiento lineal.

En conclusión, el análisis empírico de la implementación `MyQueue` es coherente con el análisis teórico. Los métodos `enqueue`, `dequeue` y `front` exhiben un comportamiento contenido y consistente con tiempo constante, mientras que `delete` presenta un crecimiento pronunciado compatible con tiempo lineal.

5.3.3. Análisis de `List` singly sin cola

La Figura 3 muestra el comportamiento empírico de los métodos evaluados sobre la implementación de lista simplemente enlazada sin cola. Esta gráfica permite observar cómo, dentro de una misma estructura, existen operaciones que se mantienen contenidas incluso

para tamaños grandes y otras cuyo tiempo de ejecución aumenta de manera pronunciada conforme crece n.

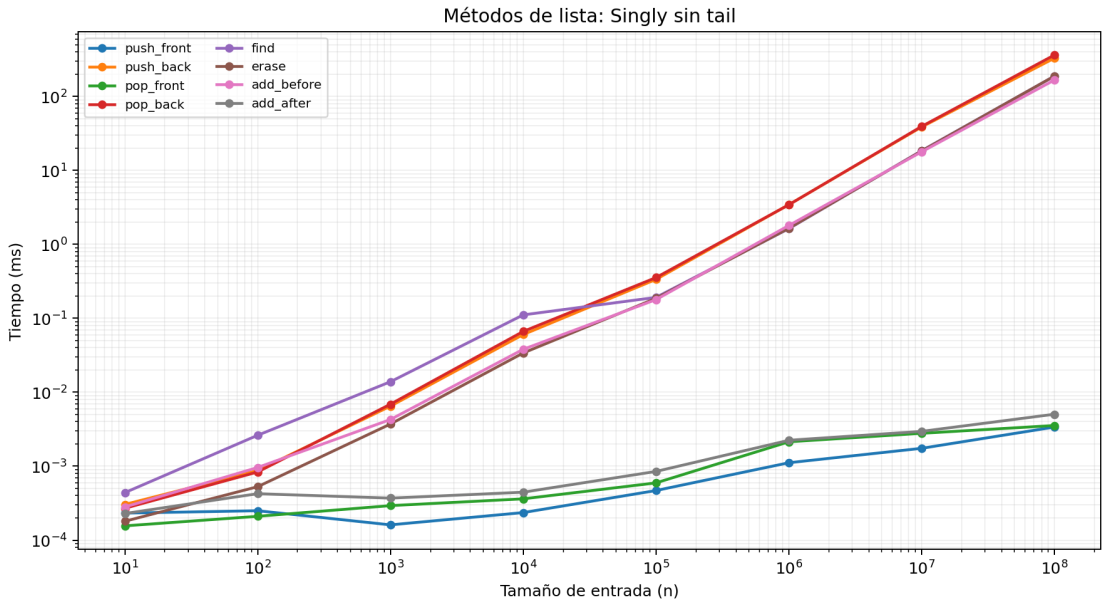

Figura 3: Métodos evaluados en la implementación `List` singly sin cola.

En la parte inferior de la gráfica se ubican los métodos `push_front`, `pop_front` y `add_after`. Estos métodos permanecen dentro del orden de cientos o pocos miles de nanosegundos a lo largo de todos los tamaños medidos, lo cual indica un comportamiento empírico contenido. En términos de mediana, `push_front` pasa de 232 ns en $n = 10$ a 3384 ns en $n = 10^8$, `pop_front` pasa de 156 ns a 3552 ns, y `add_after` pasa de 229 ns a 5048 ns. Aunque estas operaciones presentan un incremento moderado en tamaños extremos, su crecimiento sigue siendo muy inferior al de los métodos más costosos de la misma estructura.

La Tabla 7 resume las medianas obtenidas para cada operación. Se utiliza la mediana como estadístico principal porque representa mejor el comportamiento típico de cada método y reduce la influencia de variaciones puntuales del entorno de ejecución.

Cuadro 7: Tiempos medianos de ejecución para la implementación `List` singly sin cola.

Tamaño n	**push_front**	**push_back**	**pop_front**	**pop_back**
10	232	304	156	270
100	250	875	210	835
1000	161	6488	293	6898
10000	236	60303	362	66484
100000	469	336023	595	355749
1000000	1114	3441580	2137	3435910
10000000	1746	38707343	2785	39381423
100000000	3384	329706233	3552	363989199

Tamaño n	**find**	**erase**	**add_before**	**add_after**
10	439	180	281	229
100	2619	528	961	425
1000	13875	3717	4279	370
10000	111277	33977	38103	445
100000	192020	186512	179037	850
1000000	1632200	1649033	1800838	2242
10000000	18492649	18345969	17845698	2959
100000000	188417229	188733028	167061207	5048

En contraste con las operaciones anteriores, los métodos `push_back`, `pop_back`, `find`, `erase` y `add_before` muestran un crecimiento mucho más marcado. En todos estos casos, las medianas pasan de cientos o miles de nanosegundos en tamaños pequeños a decenas o cientos de millones de nanosegundos en 10^8 elementos. Por ejemplo, `push_back` alcanza 329706233 ns, `pop_back` alcanza 363989199 ns, `find` alcanza 188417229 ns, `erase` alcanza 188733028 ns y `add_before` alcanza 167061207 ns. La diferencia entre estos valores y los obtenidos por `push_front`, `pop_front` y `add_after` permite identificar claramente dos grupos de comportamiento dentro de la estructura.

Este resultado es coherente con la forma en que está organizada una lista simplemente enlazada sin cola. Las operaciones realizadas al frente de la lista, como `push_front` y `pop_front`, solo requieren modificar la referencia de la cabeza, por lo que su costo se mantiene contenido. De manera similar, `add_after` solo necesita acceder al nodo objetivo ya localizado y reasignar un número constante de referencias, lo que explica que también permanezca en la zona baja de la gráfica.

Por otro lado, las operaciones `push_back` y `pop_back` requieren recorrer la lista para llegar al final o al penúltimo nodo, ya que esta implementación no dispone de una referencia

directa al último elemento. Lo mismo ocurre con `find`, que necesariamente debe recorrer la lista hasta localizar el valor buscado. En el caso de `erase` y `add_before`, aunque ambos métodos reciben una `Position`, la ausencia de una referencia al nodo anterior obliga a recorrer la estructura para encontrar dicho nodo previo, lo cual explica que su comportamiento empírico sea muy cercano al de una búsqueda lineal.

En conjunto, los resultados experimentales de la implementación `List` singly sin cola son coherentes con su estructura interna. Los métodos `push_front`, `pop_front` y `add_after` presentan un comportamiento empírico contenido, mientras que `push_back`, `pop_back`, `find`, `erase` y `add_before` muestran un crecimiento pronunciado conforme aumenta el tamaño de la lista.

5.3.4. Análisis de `List` singly con cola

La Figura 4 presenta el comportamiento empírico de los métodos evaluados sobre la implementación de lista simplemente enlazada con referencia a cola. Al igual que en la implementación sin cola, la gráfica permite distinguir dos grupos de operaciones: aquellas que permanecen contenidas a lo largo de todos los tamaños de entrada y aquellas cuyo tiempo crece de manera pronunciada conforme aumenta n.

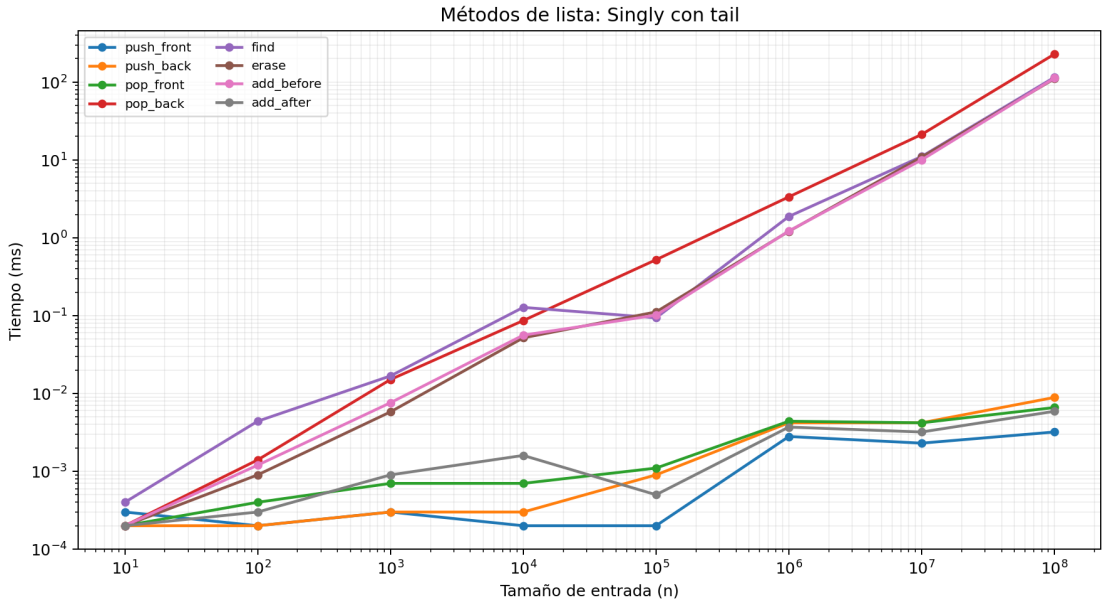

Figura 4: Métodos evaluados en la implementación `List` singly con cola.

En la parte baja de la gráfica se ubican principalmente los métodos `push_front`, `push_back`, `pop_front` y `add_after`. Estas operaciones permanecen en el orden de cientos o pocos miles de nanosegundos incluso para los tamaños más grandes. En términos de mediana, `push_front` pasa de 300 ns en $n = 10$ a 3200 ns en $n = 10^8$, `push_back` pasa de 200 ns a 8900 ns, `pop_front` pasa de 200 ns a 6600 ns, y `add_after` pasa de 200 ns a 5900 ns.

Aunque estas curvas presentan incrementos moderados en los tamaños extremos, siguen ubicadas en una escala claramente inferior a la de las operaciones más costosas.

La Tabla 8 resume las medianas obtenidas para cada operación. Como en las demás estructuras, se utiliza la mediana como estadístico principal porque ofrece una representación más estable del comportamiento típico de cada método.

Cuadro 8: Tiempos medianos de ejecución para la implementación `List` singly con cola.

Tamaño n	**push_front**	**push_back**	**pop_front**	**pop_back**
10	300	200	200	200
100	200	200	400	1400
1000	300	300	700	15100
10000	200	300	700	86400
100000	200	900	1100	522400
1000000	2800	4200	4400	3354400
10000000	2300	4200	4200	21249400
100000000	3200	8900	6600	228969000

Tamaño n	**find**	**erase**	**add_before**	**add_after**
10	400	200	200	200
100	4400	900	1200	300
1000	16800	5800	7600	900
10000	127700	51800	56100	1600
100000	93700	111600	101400	500
1000000	1878900	1211600	1223000	3700
10000000	11068100	10870700	10007900	3200
100000000	115333700	110339800	112434700	5900

En contraste con las operaciones anteriores, los métodos `pop_back`, `find`, `erase` y `add_before` muestran un crecimiento mucho más fuerte. En todos estos casos, las medianas pasan de cientos o miles de nanosegundos en tamaños pequeños a decenas o cientos de millones de nanosegundos en 10^8 elementos. Por ejemplo, `pop_back` alcanza 228969000 ns, `find` alcanza 115333700 ns, `erase` alcanza 110339800 ns y `add_before` alcanza 112434700 ns. Esta diferencia respecto a las operaciones contenidas permite identificar con claridad dos patrones de comportamiento dentro de la estructura.

El efecto principal de incorporar una referencia a cola se observa en `push_back`. A diferencia de la lista simply sin cola, donde insertar al final requería recorrer toda la estructura, en esta implementación la existencia de `tail` permite enlazar directamente el nuevo nodo al último elemento y actualizar la referencia final. Esto explica que `push_back` pase a

comportarse empíricamente como una operación contenida, claramente separada de las curvas lineales.

Sin embargo, la referencia a cola no resuelve todas las operaciones del extremo final. El método `pop_back` continúa mostrando crecimiento pronunciado porque, aunque existe acceso directo al último nodo, sigue siendo necesario recorrer la lista para localizar el penúltimo nodo antes de actualizar `tail`. Por esta razón, su comportamiento experimental sigue siendo claramente más costoso que el de `pop_front` y `push_back`.

Algo similar ocurre con `erase` y `add_before`. Aunque ambas operaciones reciben una `Position`, en una lista simplemente enlazada no existe acceso directo al nodo anterior, por lo que internamente es necesario recorrer la estructura para encontrarlo. Esto explica que sus curvas se mantengan cercanas a las de `find`, todas con crecimiento pronunciado conforme aumenta n.

Por otro lado, `add_after` permanece contenida porque, dado el nodo objetivo ya localizado, solo requiere reasignar un número constante de referencias. La comparación entre `add_after` y `add_before` ilustra de manera clara cómo la disponibilidad o ausencia de acceso al nodo previo determina el costo empírico de la operación en esta implementación.

En conjunto, los resultados experimentales de la implementación `List` singly con cola son coherentes con su estructura interna. La incorporación de una referencia al último nodo mejora claramente el comportamiento de `push_back`, pero no elimina el costo lineal de operaciones que siguen requiriendo acceso al nodo anterior o al penúltimo nodo. En consecuencia, `push_front`, `push_back`, `pop_front` y `add_after` presentan un comportamiento empírico contenido, mientras que `pop_back`, `find`, `erase` y `add_before` muestran un crecimiento pronunciado con el tamaño de la lista.

5.3.5. Análisis de `List` doubly sin cola

La Figura 5 presenta el comportamiento empírico de los métodos evaluados sobre la implementación de lista doblemente enlazada sin referencia a cola. Esta representación permite identificar con claridad dos grupos de operaciones: aquellas cuyo tiempo se mantiene contenido incluso para tamaños grandes y aquellas cuyo crecimiento se vuelve mucho más pronunciado conforme aumenta n.

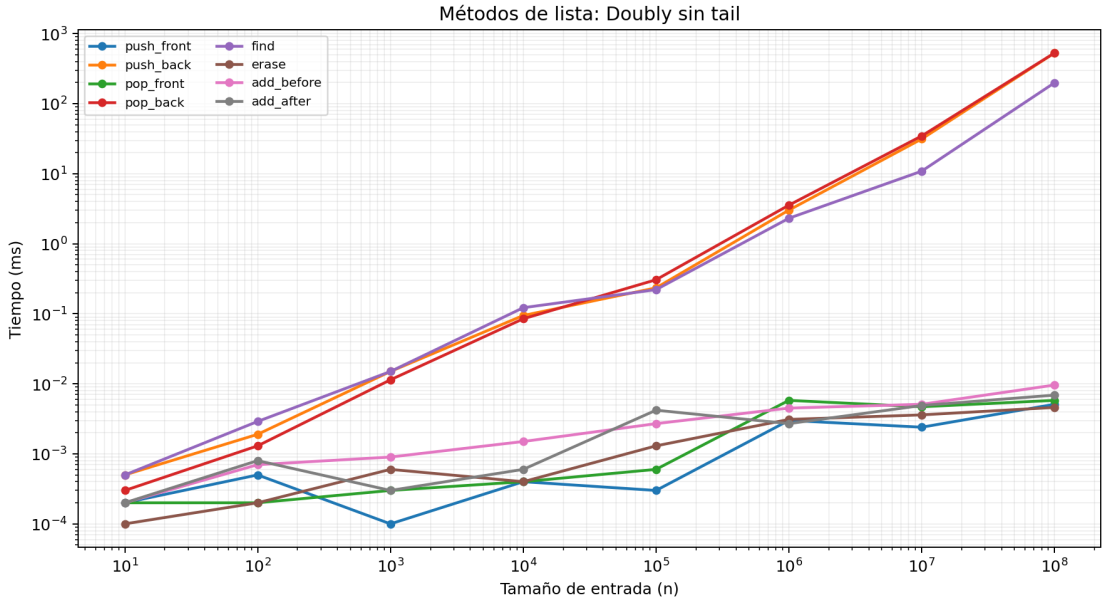

Figura 5: Métodos evaluados en la implementación `List` doubly sin cola.

En la zona inferior de la gráfica se ubican principalmente `push_front`, `pop_front`, `erase`, `add_before` y `add_after`. En todos estos casos, los tiempos medianos permanecen en el orden de cientos o pocos miles de nanosegundos incluso en los tamaños más grandes. En particular, `push_front` pasa de 200 ns en $n = 10$ a 5100 ns en $n = 10^8$, `pop_front` pasa de 200 ns a 5800 ns, `erase` pasa de 100 ns a 4600 ns, `add_before` pasa de 200 ns a 9600 ns y `add_after` pasa de 200 ns a 6900 ns. Aunque se observa un incremento moderado en tamaños extremos, todas estas operaciones permanecen muy por debajo de las curvas de mayor crecimiento.

La Tabla 9 resume las medianas obtenidas para cada uno de los métodos evaluados. Como en las demás estructuras, se utiliza la mediana como estadístico principal por ser una medida más estable del comportamiento típico de las operaciones.

Cuadro 9: Tiempos medianos de ejecución para la implementación `List` doubly sin cola.

Tamaño n	**push_front**	**push_back**	**pop_front**	**pop_back**
10	200	500	200	300
100	500	1900	200	1300
1000	100	15000	300	11400
10000	400	94500	400	84800
100000	300	234500	600	306100
1000000	3000	3020200	5800	3554600
10000000	2400	31293600	4700	34592400
100000000	5100	528790900	5800	529549900

Tamaño n	**find**	**erase**	**add_before**	**add_after**
10	500	100	200	200
100	2900	200	700	800
1000	15000	600	900	300
10000	122100	400	1500	600
100000	221000	1300	2700	4200
1000000	2310900	3100	4500	2700
10000000	10868900	3600	5100	4900
100000000	198752600	4600	9600	6900

En contraste con el grupo anterior, los métodos `push_back`, `pop_back` y `find` muestran un crecimiento mucho más pronunciado. En los tres casos, las medianas pasan de cientos o miles de nanosegundos en tamaños pequeños a decenas o cientos de millones de nanosegundos en 10^8 elementos. Específicamente, `push_back` alcanza 528790900 ns, `pop_back` alcanza 529549900 ns y `find` alcanza 198752600 ns. La separación entre estas curvas y las operaciones contenidas permite distinguir con claridad qué métodos están dominados por recorridos lineales de la estructura.

El comportamiento de esta implementación es coherente con su organización interna. Al tratarse de una lista doblemente enlazada, cada nodo dispone de referencias tanto al siguiente como al anterior. Gracias a ello, operaciones como `erase`, `add_before` y `add_after`, una vez conocida la posición objetivo, solo requieren reasignar un número constante de referencias. Esta es la razón principal por la que dichas operaciones permanecen contenidas y muy por debajo de las curvas lineales.

Asimismo, `push_front` y `pop_front` continúan siendo operaciones contenidas porque se realizan directamente sobre la cabeza de la lista. Sin embargo, la ausencia de una referencia a cola afecta de forma importante a `push_back` y `pop_back`. Aunque la lista sea

doblemente enlazada, si no existe acceso directo al último nodo sigue siendo necesario recorrer la estructura para alcanzar el final o el penúltimo nodo, lo que explica el fuerte crecimiento empírico de ambas operaciones.

El método `find` también presenta un crecimiento pronunciado, como es esperable, ya que necesariamente debe recorrer nodos sucesivos hasta localizar el valor buscado. De hecho, su curva comparte el mismo patrón general de incremento que `push_back` y `pop_back`, aunque con una constante menor que las operaciones asociadas al extremo final de la lista.

Un aspecto particularmente importante de esta implementación es la diferencia entre `find` y las operaciones basadas en una `Position`. Mientras `find` crece de forma clara con el tamaño de la lista, `erase`, `add_before` y `add_after` permanecen contenidas. Esto confirma experimentalmente que, en una lista doblemente enlazada, disponer de acceso directo al nodo objetivo y a su referencia previa elimina la necesidad de un recorrido adicional para efectuar la modificación.

En conjunto, los resultados experimentales de la implementación `List` doubly sin cola son consistentes con su estructura interna. Los métodos `push_front`, `pop_front`, `erase`, `add_before` y `add_after` muestran un comportamiento empírico contenido, mientras que `push_back`, `pop_back` y `find` presentan un crecimiento claramente más pronunciado al aumentar el tamaño de la lista.

5.3.6. Análisis de `List` doubly con cola

La Figura 6 presenta el comportamiento empírico de los métodos evaluados sobre la implementación de lista doblemente enlazada con referencia a cola. A diferencia de las implementaciones anteriores, en esta estructura la mayoría de las operaciones permanece agrupada en la zona inferior de la gráfica, mientras que únicamente `find` se separa de manera clara y exhibe un crecimiento pronunciado conforme aumenta n.

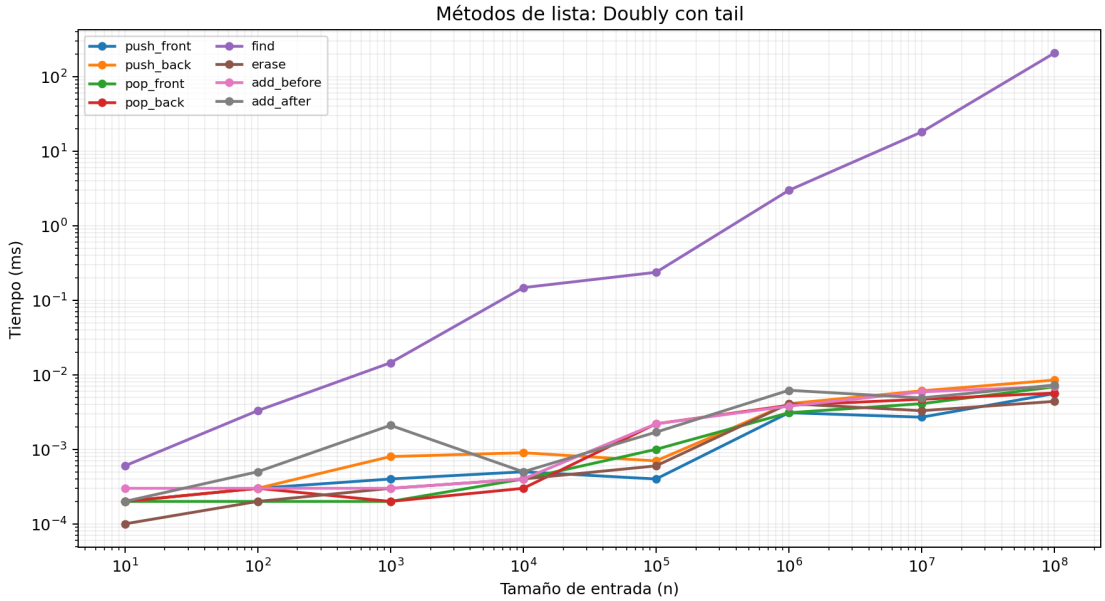

Figura 6: Métodos evaluados en la implementación `List` doubly con cola.

En la zona inferior de la gráfica se encuentran `push_front`, `push_back`, `pop_front`, `pop_back`, `erase`, `add_before` y `add_after`. Todas estas operaciones se mantienen en el orden de cientos o pocos miles de nanosegundos a lo largo de los distintos tamaños de entrada, incluso en el caso extremo de 10^8 elementos. En términos de mediana, `push_front` pasa de 200 ns en $n = 10$ a 5600 ns en $n = 10^8$, `push_back` pasa de 200 ns a 8500 ns, `pop_front` pasa de 200 ns a 6900 ns, `pop_back` pasa de 200 ns a 5700 ns, `erase` pasa de 100 ns a 4400 ns, `add_before` pasa de 300 ns a 7000 ns y `add_after` pasa de 200 ns a 7300 ns. Aunque algunas de estas curvas muestran incrementos moderados en tamaños grandes, todas permanecen muy contenidas frente a la operación de mayor costo.

La Tabla 10 resume las medianas obtenidas para cada método. Como en las demás estructuras, se utiliza la mediana como estadístico principal porque representa mejor el comportamiento típico de cada operación y reduce la influencia de variaciones puntuales.

Cuadro 10: Tiempos medianos de ejecución para la implementación `List` doubly con cola.

Tamaño n	**push_front**	**push_back**	**pop_front**	**pop_back**
10	200	200	200	200
100	300	300	200	300
1000	400	800	200	200
10000	500	900	400	300
100000	400	700	1000	2200
1000000	3100	4100	3100	3900
10000000	2700	6100	4100	4700
100000000	5600	8500	6900	5700

Tamaño n	**find**	**erase**	**add_before**	**add_after**
10	600	100	300	200
100	3300	200	300	500
1000	14500	300	300	2100
10000	147900	400	400	500
100000	237500	600	2200	1700
1000000	2988200	4100	3800	6200
10000000	18129900	3300	5900	4900
100000000	208045700	4400	7000	7300

La operación que se separa claramente del resto es `find`. Sus medianas pasan de 600 ns en $n = 10$ a 208045700 ns en $n = 10^8$, lo que muestra un crecimiento muy superior al de los demás métodos. Este resultado es completamente consistente con la estructura interna de la implementación: localizar un valor específico sigue requiriendo recorrer la lista nodo por nodo hasta alcanzar el elemento buscado.

Por el contrario, la combinación de doble enlace y referencia a cola beneficia claramente al resto de las operaciones. Las operaciones en el frente, como `push_front` y `pop_front`, continúan siendo contenidas porque actúan directamente sobre la cabeza de la lista. Las operaciones en el extremo final, como `push_back` y `pop_back`, también permanecen contenidas gracias a la existencia de una referencia directa al último nodo y al acceso al nodo previo mediante el enlace doble. Esto elimina la necesidad de recorrer la estructura para insertar o eliminar al final.

De manera similar, `erase`, `add_before` y `add_after` se mantienen en la zona baja de la gráfica porque, una vez conocida la `Position` objetivo, la modificación de enlaces se realiza mediante un número constante de reasignaciones. La diferencia frente a las listas simplemente enlazadas es clara: aquí no es necesario buscar explícitamente el nodo

anterior, ya que cada nodo lo conoce directamente a través de la referencia `prev`.

En esta implementación, por tanto, el único método cuyo costo sigue estando dominado por un recorrido lineal es `find`. Esto convierte a la lista doblemente enlazada con cola en la versión más equilibrada entre las cuatro implementaciones analizadas, ya que concentra la mayor parte de sus operaciones fundamentales en una misma banda de tiempos relativamente bajos.

En conjunto, los resultados experimentales son coherentes con la estructura interna de la implementación `List` doubly con cola. Los métodos `push_front`, `push_back`, `pop_front`, `pop_back`, `erase`, `add_before` y `add_after` muestran un comportamiento empírico contenido, mientras que `find` se mantiene como la única operación con crecimiento claramente pronunciado al aumentar el tamaño de la lista.

5.3.7. Comparación transversal de operaciones de extracción

Como cierre del análisis de las listas, se realizó una comparación transversal entre operaciones equivalentes de extracción en las diferentes estructuras implementadas. En particular, se contrastaron `pop` en `MyStack<T>`, `dequeue` en `MyQueue<T>` y `pop_front` en las cuatro implementaciones de `List`. El objetivo de esta comparación no fue establecer una equivalencia estricta entre estructuras con comportamientos abstractos distintos, sino observar cómo se comportan, en la práctica, operaciones que extraen el elemento principal accesible de cada estructura.

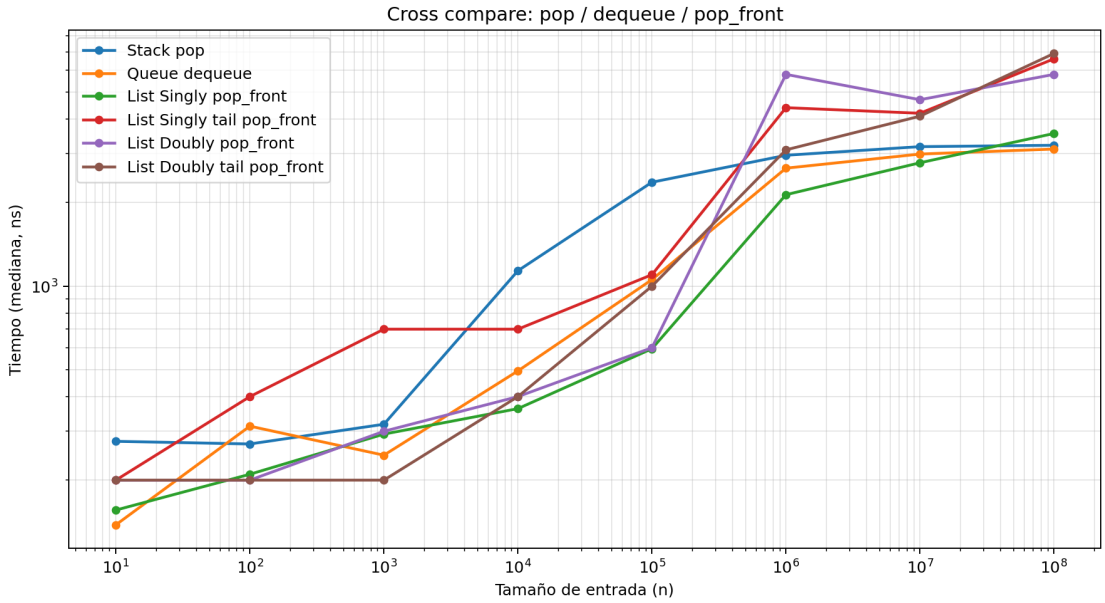

Figura 7: Comparación transversal entre `pop`, `dequeue` y `pop_front`.

La Figura 7 muestra que todas las curvas permanecen dentro de una misma escala de tiempo y muy por debajo del crecimiento observado en operaciones lineales estudiadas

previamente. Esto indica que, desde el punto de vista empírico, `pop`, `dequeue` y `pop_front` se comportan como operaciones contenidas y compatibles con tiempo constante en las implementaciones analizadas.

Aun así, la gráfica también deja ver diferencias de constante entre estructuras. En tamaños pequeños, `dequeue` y `pop_front` en las listas singly tienden a presentar los menores tiempos. Por su parte, `pop` en la pila se mantiene en una franja ligeramente superior, mientras que las implementaciones doubly, especialmente a partir de tamaños grandes, muestran tiempos medianos algo mayores. Estas diferencias no cambian la naturaleza asintótica de las operaciones, pero sí evidencian que la estructura concreta y el número de referencias actualizadas en cada operación influyen en el costo real observado.

Un aspecto importante de la comparación es que ninguna de las curvas presenta el patrón de crecimiento pronunciado característico de operaciones lineales. Incluso cuando algunas implementaciones muestran incrementos a partir de 10^6 elementos, dichos aumentos siguen ocurriendo dentro de una misma banda de tiempo, del orden de los miles de nanosegundos. Esto refuerza la idea de que la diferencia principal entre estas operaciones no está en su orden de complejidad, sino en sus constantes de implementación y en el efecto práctico del entorno de ejecución sobre estructuras muy grandes.

También resulta interesante que las cuatro implementaciones de listas no se comporten exactamente igual entre sí. Las listas singly y singly con cola permanecen algo más contenidas en los tamaños extremos, mientras que las implementaciones doubly muestran tiempos ligeramente superiores. Esto sugiere que, aunque el doble enlace aporta ventajas claras en operaciones como `erase`, `add_before` y `add_after`, en una operación tan simple como `pop_front` puede introducir un costo constante mayor debido al mantenimiento adicional de referencias.

En conjunto, esta comparación transversal permite cerrar el análisis empírico con una visión global: estructuras distintas pueden ofrecer operaciones equivalentes con el mismo orden de complejidad, aunque no necesariamente con la misma constante de tiempo. La gráfica confirma que `MyStack<T>`, `MyQueue<T>` y las diferentes implementaciones de `List` resuelven la extracción del elemento principal de forma empíricamente contenida, mientras que las diferencias observadas entre curvas responden principalmente al diseño interno de cada estructura.

6. Conclusiones

- El desarrollo del proyecto permitió comprobar que el análisis de complejidad no se queda en el plano teórico, sino que termina reflejándose en el comportamiento real de las estructuras cuando estas se implementan y se miden de forma sistemática. Las

diferencias entre listas simply, listas con cola, listas doubly y listas doubly con cola no fueron solo conceptuales, sino que se hicieron visibles en los tiempos de ejecución observados.

- Una parte importante del trabajo consistió en ajustar la metodología experimental. Durante las primeras corridas aparecieron resultados poco consistentes, especialmente en operaciones $O(1)$, lo que llevó a revisar no solo las implementaciones, sino también la forma de ejecutar los benchmarks. Finalmente, al correrlos método a método y medir cada operación sobre estructuras previamente construidas hasta tamaño n, se obtuvieron resultados mucho más estables y coherentes.

- También se concluyó que, para este tipo de mediciones, la mediana resulta más representativa que el promedio. Aunque ambos valores fueron registrados, la mediana describió mejor el comportamiento típico de cada operación y redujo el efecto de variaciones puntuales del entorno de ejecución, especialmente en operaciones de muy corta duración.

- En `MyStack<T>` y `MyQueue<T>`, las operaciones fundamentales de inserción, extracción y consulta se mantuvieron contenidas incluso para tamaños grandes, mientras que `delete` mostró un crecimiento mucho más pronunciado. Esto confirmó experimentalmente la diferencia entre operaciones de costo constante y operaciones cuyo tiempo depende de un recorrido interno de la estructura.

- En las listas enlazadas, los resultados dejaron ver con claridad cómo la organización interna determina el costo de los métodos. En la lista simply sin cola, las operaciones sobre el frente y `add_after` se mantuvieron contenidas, mientras que las operaciones asociadas al final, la búsqueda y aquellas que requieren ubicar el nodo anterior crecieron de forma mucho más marcada. Esto confirmó el costo adicional que introduce la ausencia de una referencia al final y del acceso directo al nodo previo.

- Al incorporar una referencia a cola en la lista simply, se observó una mejora clara en `push_back`, que dejó de comportarse como una operación costosa. Sin embargo, `pop_back`, `erase` y `add_before` siguieron mostrando crecimiento pronunciado, lo que evidenció que contar con el último nodo no resuelve por sí solo los problemas derivados de no poder acceder directamente al nodo anterior.

- En la lista doubly sin cola, el doble enlace permitió que `erase`, `add_before` y `add_after` se mantuvieran contenidas, ya que una vez conocida la posición objetivo las modificaciones pueden hacerse sin recorridos adicionales. No obstante, `push_back` y `pop_back` continuaron siendo costosas debido a la falta de una referencia directa al último nodo.

- La implementación doubly con cola fue la más equilibrada de todas. En ella, casi

todas las operaciones fundamentales permanecieron dentro de una misma banda de tiempos bajos y `find` quedó como la única operación claramente dominada por un recorrido lineal. Desde el punto de vista experimental, esta implementación fue la que mejor aprovechó la combinación de enlace doble y referencia al último nodo.

- Las comparaciones entre estructuras también reforzaron la interpretación de los resultados. Al contrastar operaciones equivalentes, como `pop`, `dequeue` y `pop_front`, se observó que, aunque existen diferencias de constante entre implementaciones, el comportamiento asintótico general se conserva. Esto permitió confirmar que la complejidad depende tanto de la operación como de la estructura sobre la que se implementa.
- En conjunto, el proyecto permitió establecer una relación clara entre diseño interno, complejidad teórica y comportamiento empírico. Más allá de los tiempos concretos obtenidos, el principal aprendizaje fue que pequeños cambios en la representación de una estructura, como agregar una cola o disponer de enlaces dobles, pueden transformar de forma significativa el costo real de sus operaciones más importantes.

7. Repositorio

El repositorio del proyecto se encuentra disponible en el siguiente enlace:

Repositorio en GitHub